

C++ Standard Library Issues Resolved Directly In Jacksonville

Doc. no. P0304R0

Date: Revised 2016-03-04 at 17:03:35 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Immediate Issues

[2296](#). `std::addressof` should be `constexpr`

Section: 20.7.12.1 [specialized.addressof] **Status:** [Immediate](#) **Submitter:** Daryle Walker **Opened:** 2013-09-08 **Last modified:** 2016-03-04

Priority: 3

View all other [issues](#) in [specialized.addressof].

Discussion:

I'm writing a function that needs to be `constexpr` and I wanted to take the address of its input. I was thinking of using `std::addressof` to be safe, but it isn't currently `constexpr`. A [sample implementation](#) couldn't be `constexpr` under the C++11 rules, though.

Daniel Krügler:

Indeed the core language clarified by [CWG 1312](#) and by [CWG 1384](#), that such emulations of `std::addressof` implementations are not valid in constant expressions, therefore it seems more like a defect than a feature request to ask for the guarantee that `std::addressof` is a `constexpr` function. It should be added that a similar requirement already exists for `offsetof` indirectly via the C99 standard as of 7.17 p3:

The macros are [...]

```
offsetof(type, member-designator)
```

which expands to an integer constant expression that has type `size_t` [...]

combined with the noted property in C++11 that:

"`offsetof` is required to work as specified even if unary operator`&` is overloaded for any of the types involved"

Therefore implementations should already be able without heroic efforts to realize this functionality by some intrinsic. The wording needs at least to ensure that for any lvalue core constant expression *e* the expression `std::addressof(e)` is a core constant expression.

[2013-09 Chicago]

[2014-06-08, Daniel improves wording]

It has been ensured that the wording is in sync with the recent working paper and the usage of "any" has been improved to say "every" instead (the fix is similar to that applied by LWG [2150](#)).

Previous resolution from Daniel [SUPERSEDED]:

1. Change header `<memory>` synopsis, 20.7.2 [memory.syn] as indicated:

```
namespace std {  
    [...]  
    // 20.7.12 [specialized.algorithms], specialized algorithms:  
    template <class T> constexpr T* addressof(T& r) noexcept;  
    [...]
```

```
}
```

2. Change 20.7.12.1 [specialized.addressof] as indicated:

```
template <class T> constexpr T* addressof(T& r) noexcept;
```

-1- *Returns*: The actual address of the object or function referenced by *r*, even in the presence of an overloaded operator&.

-?- *Remarks*: For every lvalue core constant expression *e* (5.20 [expr.const]), the expression `std::addressof(e)` is a core constant expression.

[2014-06-09, further improvements]

A new wording form is now used similar to the approach used by LWG [2234](#), which is a stricter way to impose the necessary implementation requirements.

[2015-05, Lenexa]

STL: the intent of this change is good; I think the wording is good

- I'm a bit worried about asking for a compiler hook

- if every implementer says: yes they can do it we should be good

EB: there is missing the word "a" before "subexpression" (in multiple places)

MC: the editor should do - we rely on our editors

MC: move to Review with a note stating that we wait for implementation experience first

- in favor: 13, opposed: 0, abstain: 2 HB: N4430 will bring something which is addressing this issue

MC: good we didn't go to ready then

Proposed resolution:

This wording is relative to N3936.

1. Introduce the following new definition to the existing list in 17.3 [definitions]: [*Drafting note*: If LWG [2234](#) is accepted before this issue, the accepted wording for the new definition should be used instead — *end drafting note*]

constant subexpression [defns.const.subexpr]

an expression whose evaluation as a subexpression of a *conditional-expression CE* (5.16 [expr.cond]) would not prevent *CE* from being a core constant expression (5.20 [expr.const]).

2. Change header <memory> synopsis, 20.7.2 [memory.syn] as indicated:

```
namespace std {  
    [...]  
    // 20.7.12 [specialized.algorithms], specialized algorithms:  
    template <class T> constexpr T* addressof(T& r) noexcept;  
    [...]  
}
```

3. Change 20.7.12.1 [specialized.addressof] as indicated:

```
template <class T> constexpr T* addressof(T& r) noexcept;
```

-1- *Returns*: The actual address of the object or function referenced by *r*, even in the presence of an overloaded operator&.

-?- *Remarks*: An expression `std::addressof(E)` is a constant subexpression (17.3.28 [defns.const.subexpr]), if *E* is an lvalue constant subexpression.